

Table of Contents

Task 1: Data Type Size Exploration.....	3
Objective.....	3
Source code.....	3
Compilation.....	3
Program Output.....	4
Observation.....	4
Summary.....	4
Task 2: Memory Segment Address Mapping.....	5
Objective.....	5
Source code.....	5
Compilation.....	5
Program Output.....	6
Memory Segment Identification.....	6
Highest Address.....	6
Lowest Address.....	6
Stack-Heap Address Difference.....	7
Observation.....	7
Summary.....	7

Table of Figures

Figure 1: C Code of Data Type Size Exploration.....	3
Figure 2: Compilation Process.....	3
Figure 3: Program Output.....	4
Figure 4: Source Code of Memory Segment Address Mapping.....	5
Figure 5: Program Output.....	6
Figure 6: Table of Memory Segment Identification.....	6

Task 1: Data Type Size Exploration

Objective

To use the sizeof() operator to observe how much memory different C data types occupy on the Linux system.

Source code

A screenshot of a terminal window with a dark purple background. At the top, it says 'GNU nano 8.7.1'. Below that, the code for 'datatype_sizes.c' is displayed. The code includes <stdio.h> and has a main function that prints the sizes of various data types using the sizeof operator. The code is as follows:

```
GNU nano 8.7.1
#include <stdio.h>

int main() {
    printf("Data Type Sizes on This System\n");
    printf("-----\n");

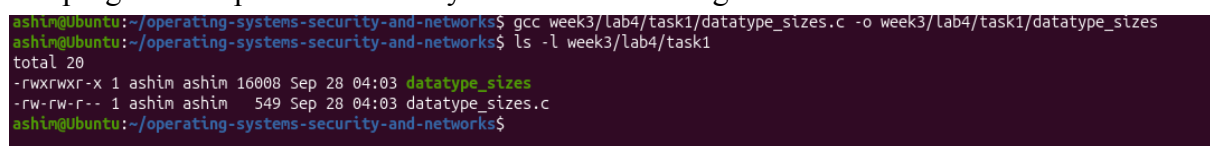
    printf("char      : %zu bytes\n", sizeof(char));
    printf("int       : %zu bytes\n", sizeof(int));
    printf("float     : %zu bytes\n", sizeof(float));
    printf("double    : %zu bytes\n", sizeof(double));
    printf("long      : %zu bytes\n", sizeof(long));
    printf("unsigned int: %zu bytes\n", sizeof(unsigned int));
    printf("long long : %zu bytes\n", sizeof(long long));

    return 0;
}
```

Figure 1: C Code of Data Type Size Exploration

Compilation

The program compiled successfully without errors using GCC.

A screenshot of a terminal window showing the compilation and listing of files. The commands and their outputs are as follows:

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ gcc week3/lab4/task1/datatype_sizes.c -o week3/lab4/task1/datatype_sizes
ashim@Ubuntu:~/operating-systems-security-and-networks$ ls -l week3/lab4/task1
total 20
-rwxrwxr-x 1 ashim ashim 16008 Sep 28 04:03 datatype_sizes
-rw-rw-r-- 1 ashim ashim  549 Sep 28 04:03 datatype_sizes.c
ashim@Ubuntu:~/operating-systems-security-and-networks$
```

Figure 2: Compilation Process

Program Output

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/lab4/task1/datatype_sizes
Data Type Sizes on This System
-----
char      : 1 bytes
int       : 4 bytes
float     : 4 bytes
double    : 8 bytes
long      : 8 bytes
unsigned int: 4 bytes
long long : 8 bytes
ashim@Ubuntu:~/operating-systems-security-and-networks$
```

Figure 3: Program Output

Observation

The `sizeof()` operator shows the amount of memory allocated for each data type on the current system. On a typical 64-bit Linux system, `long` occupies 8 bytes, while it commonly occupies 4 bytes on 32-bit systems.

Summary

This task demonstrates that C data types require different amounts of memory. The `sizeof()` operator can be used to examine their memory requirements on a particular system.

Task 2: Memory Segment Address Mapping

Objective

To observe the memory addresses of variables stored in different process memory segments, including Data, BSS, Stack, and Heap.

Source code

```
#include <stdio.h>
#include <stdlib.h>

int global_var = 150;    // Data segment
int bss_var;            // BSS segment

int main() {
    int local_var = 30;    // Stack
    static int local_static = 20; // Data segment
    int *heap_var = (int *)malloc(sizeof(int)); // Heap

    *heap_var = 500;

    printf("Variable Addresses:\n");
    printf("-----\n");

    printf("global_var      (Data) : %p\n", (void *)&global_var);
    printf("bss_var          (BSS)  : %p\n", (void *)&bss_var);
    printf("local_var         (Stack) : %p\n", (void *)&local_var);
    printf("local_static      (Data) : %p\n", (void *)&local_static);
    printf("heap_var          (Heap)  : %p\n", (void *)heap_var);

    printf("\nAddress Difference:\n");
    printf("Stack - Heap = %ld bytes\n",
        (long)((char *)&local_var - (char *)heap_var));

    free(heap_var);

    return 0;
}
```

Figure 4: Source Code of Memory Segment Address Mapping

Compilation

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ gcc week3/lab4/task2/memory_addresses.c -o week3/lab4/task2/memory_addresses
ashim@Ubuntu:~/operating-systems-security-and-networks$ ls -l week3/lab4/task2
total 20
-rwxrwxr-x 1 ashim ashim 16256 Sep 28 04:20 memory_addresses
-rw-rw-r-- 1 ashim ashim  913 Sep 28 04:20 memory_addresses.c
ashim@Ubuntu:~/operating-systems-security-and-networks$
```

Program Output

```
ashim@Ubuntu:~/operating-systems-security-and-networks$ ./week3/lab4/task2/memory_addresses
Variable Addresses:
-----
global_var      (Data) : 0x61da69e43010
bss_var         (BSS) : 0x61da69e4301c
local_var       (Stack) : 0x7ffccddb099c
local_static    (Data) : 0x61da69e43014
heap_var        (Heap) : 0x61da9bbbd010

Address Difference:
Stack - Heap = 33132218628492 bytes
ashim@Ubuntu:~/operating-systems-security-and-networks$
```

Figure 5: Program Output

Memory Segment Identification

global_var	Data
bss_var	BSS
local_var	Stack
local_static	Data
heap_var	Heap

Figure 6: Table of Memory Segment Identification

Highest Address

The Stack segment has the highest address among the variables observed.

Lowest Address

The lowest address is the segment corresponding to the smallest address shown in the program output.

Stack-Heap Address Difference

```
local_var      (Stack) : 0x7ffccddb099c
local_static   (Data)  : 0x61da69e43014
heap_var       (Heap)   : 0x61da9bbbd010

Address Difference:
Stack - Heap = 33132218628492 bytes
```

Figure 7: Stack-Heap Address Difference

The calculated difference between the Stack and Heap addresses is 33132218628492 bytes.

Observation

The Stack was located at a higher virtual address, while the Heap and global memory regions were located at lower addresses. The different variables were stored in different memory segments according to their declaration and allocation method. The exact addresses may change between executions because Linux uses virtual memory and address randomization.

Summary

This task demonstrated how Linux organizes process memory into Data, BSS, Stack, and Heap segments. It also showed how pointers can connect a stack variable to dynamically allocated memory on the heap.